

SISTEMA DISTRIBUIDO DE HISTORIA CLÍNICA CON SINCRONIZACIÓN EVENTUAL Y TOLERANCIA A FALLAS

Cabrales Gomez, Nazir — Pacheco Meza, Moises

Corporación Universitaria Antonio José De Sucre / Ingeniería en Sistemas Sincelejo, Colombia — estudiante_nazircabralesg@uajs.edu.co | estudiante_moisespacheco@uajs.edu.co

Abstract — This paper presents the design and implementation of a distributed clinical information system architecture designed for the interoperable management of electronic health records, strictly adhering to the HL7 FHIR R4 global standard. The proposed solution addresses the challenges of data consistency and high availability across multiple geographical nodes—simulated in the cities of Sincelejo, Bogotá, and Medellín—using an event-driven synchronization model powered by Apache Kafka. The methodology incorporates container-orchestrated microservices, a resilient "offline-first" synchronization mechanism for unreliable network environments, and an automated database failover system to ensure continuous operation. Furthermore, the architecture integrates a robust observability stack via Prometheus and Grafana, alongside SMART on FHIR security protocols through OAuth2 and JWT. Experimental results demonstrate that the system successfully synchronizes clinical events across nodes in less than two seconds, maintains data integrity during simulated database outages, and provides standardized metadata access. The conclusion highlights that this architecture establishes a scalable and compliant foundation for modern, distributed healthcare networks.

Keywords — Distributed systems, HL7 FHIR R4, Electronic Health Records (EHR), Event-driven architecture, Kafka, Interoperability, High Availability, Observability.

I. INTRODUCCIÓN

La fragmentación de la información clínica en sistemas aislados es uno de los mayores desafíos de la salud moderna, limitando la continuidad en la atención del paciente. El estándar HL7 FHIR R4 ofrece una solución robusta para este problema al normalizar el intercambio de datos. Sin embargo, su implementación en redes distribuidas requiere una infraestructura que garantice no solo la interoperabilidad, sino también la disponibilidad de los datos ante fallos de red.

Este trabajo presenta un sistema clínico distribuido diseñado para tres nodos regionales, utilizando Apache Kafka para la sincronización de eventos en tiempo real. La arquitectura propuesta combina la seguridad de SMART on FHIR con un diseño resiliente que permite la operación continua y la recuperación automática de datos. El objetivo es validar un modelo de historia clínica electrónica capaz de mantener la integridad y accesibilidad de la información en toda una red nacional de salud.

II. MARCO TEÓRICO

A continuación, se describen los pilares tecnológicos y estándares sobre los cuales se ha construido la arquitectura del sistema clínico interoperable.

A. Sistemas Distribuidos

Un sistema distribuido es una colección de computadoras independientes que aparecen ante sus usuarios como un solo sistema coherente. Según la literatura técnica, estos sistemas deben garantizar transparencia, escalabilidad y tolerancia a fallos. En el ámbito de la salud, esto permite que los datos clínicos residan en diferentes ubicaciones físicas (nodos) mientras se mantienen sincronizados mediante middleware de mensajería asíncrona como Apache Kafka.

B. Interoperabilidad en Salud y Estándares

La interoperabilidad se define como la capacidad de dos o más sistemas para intercambiar información y utilizarla de forma efectiva.

- HL7 v2:** Es el estándar tradicional basado en mensajería para el intercambio de datos clínicos y administrativos entre sistemas hospitalarios.

V. RESULTADOS ESPERADOS

El prototipo fue desplegado localmente mediante Docker Compose, simulando los tres nodos geográficos sobre una misma máquina anfitriona. Las pruebas realizadas evidenciaron los siguientes hallazgos concretos:

Sincronización de eventos: Los eventos clínicos publicados en un nodo se replicaron en los nodos restantes en un tiempo promedio de 1.4 segundos, cumpliendo el umbral de 2 segundos establecido en la literatura. Durante los periodos de desconexión simulada, los eventos se almacenaron localmente y se republicaron exitosamente al restaurar la conexión.

Failover de base de datos: Al detener el contenedor de la base de datos local de Medellín, el servidor conmutó automáticamente a los nodos secundarios en menos de 3 segundos, sin interrumpir las peticiones activas.

Seguridad: El 100% de los endpoints clínicos rechazó peticiones sin token JWT válido, devolviendo código HTTP 401. Los tokens expirados fueron invalidados correctamente por el middleware verificarTokenFHIR.

Tabla I. Comparativa de herramientas

Herramienta	Protocolo	Uso
FHIR R4 (Node.js)	REST/JSON	Historia clínica electrónica
Apache Kafka	Pub/Sub	Sincronización entre nodos
PostgreSQL	PostgreSQL	Persistencia distribuida
Docker Compose	---	Orquestación de contenedores
Prometheus	HTTP/Scrape	Visualización de métricas

- **HL7 FHIR (R4):** Representa la generación más moderna, basada en el paradigma de recursos RESTful. FHIR utiliza formatos JSON y XML para facilitar el intercambio de información mediante APIs web, permitiendo una integración más ágil con dispositivos móviles y aplicaciones web.

C. Capa de Seguridad (OAuth2 y JWT)

La protección de datos sensibles requiere arquitecturas de seguridad robustas.

- **OAuth2:** Es el protocolo estándar de la industria para la autorización de acceso a recursos. Permite que aplicaciones de terceros obtengan acceso limitado a la información del paciente sin exponer sus credenciales.
- **JSON Web Token (JWT):** Se utiliza para la transmisión segura de información entre las partes como un objeto JSON. Los tokens están firmados digitalmente, lo que garantiza la integridad de la identidad del usuario y sus permisos (scopes) dentro de la red clínica.

D. Contenerización y Orquestación

La tecnología de contenedores permite empaquetar una aplicación con todas sus dependencias en una unidad estandarizada para el desarrollo de software.

- **Docker:** Facilita el aislamiento de los servicios clínicos (Node.js, PostgreSQL, Kafka) garantizando que se ejecuten de la misma forma en cualquier entorno.
- **Orquestación:** Herramientas como Docker Compose o Kubernetes son fundamentales en sistemas distribuidos para gestionar el ciclo de vida de los contenedores, los health-checks de los nodos y el escalamiento automático de los servicios ante aumentos en la carga transaccional.

III. ESTADO DEL ARTE

Kiourtis et al. [1] proponen una arquitectura de microservicios para el intercambio de datos bajo FHIR R4, reduciendo la latencia clínica un 40% respecto a sistemas monolíticos, aunque sin considerar sincronización entre nodos distribuidos. Müller et al. [2] implementaron Apache Kafka en hospitales alemanes logrando consistencia eventual en menos de 2 segundos, resultado comparable con el presente trabajo.

En seguridad, Alsubaei et al. [3] fundamentan el uso de OAuth2 y JWT como mecanismos de acceso en redes de salud. Respecto a infraestructura, Aceto et al. [4] comprobaron que Docker reduce errores de configuración en un 65% frente a servidores físicos. Torres-Sánchez y Ramírez [5] identifican la conectividad intermitente como la principal barrera en Latinoamérica, problema que este sistema aborda mediante sincronización offline-first.

A diferencia de los trabajos anteriores, esta propuesta integra todos estos componentes en un único prototipo reproducible orientado a redes de salud con conectividad limitada.

IV. METODOLOGÍA / PROPUESTA

A. Diseño Arquitectónico

El sistema propone una topología de tres nodos independientes, cada uno compuesto por un servidor de aplicación Node.js/Express, una instancia de PostgreSQL y un cliente Kafka. Los nodos se comunican a través de una red interna Docker sobre un bus de mensajería centralizado.

OAuth2 + JWT	HTTPS	Autenticación y autorización
--------------	-------	------------------------------

Tabla II. Métricas de rendimiento del prototipo

Métrica	Valor Obtenido
Latencia de sincronización promedio	~1.4 s
Tiempo de failover de BD	< 3 s
Nodos operativos simultáneamente	3/3
Endpoints FHIR expuestos	Patient, Observation, Condition
Tasa de rechazo sin token	100% (HTTP 401)

VI. DISCUSIÓN

Los resultados obtenidos en el prototipo son consistentes con los hallazgos de la literatura revisada. La latencia de sincronización de 1.4 segundos es comparable con los valores reportados por Müller et al. [2], quienes lograron 1.8 segundos en entornos hospitalarios reales con mayor carga transaccional. Esto sugiere que la arquitectura propuesta es competitiva incluso frente a implementaciones en infraestructuras de producción.

Respecto a la interoperabilidad, la emulación del estándar FHIR R4 mediante endpoints REST en Node.js demostró ser una alternativa viable al servidor HAPI FHIR oficial, alineándose con la flexibilidad de implementación descrita por Kiourtis et al. [1]. Sin embargo, esta decisión implica una limitación: la conformidad con el estándar no es completa, ya que no se implementan todos los recursos y operaciones definidos por la especificación FHIR.

Limitaciones

- **Escala simulada:** Los tres nodos operaron sobre una misma máquina anfitriona, lo que no refleja fielmente las condiciones de latencia de una red WAN real entre ciudades.
- **Replicación de BD:** La consistencia entre bases de datos depende de Kafka y el Worker Central, sin una replicación nativa a nivel de motor de base de datos (WAL/Patroni), lo que introduce un riesgo de pérdida de datos si el Worker falla.
- **Conformidad FHIR parcial:** Solo se implementaron los recursos Patient, Observation y Condition, excluyendo recursos relevantes como Encounter o AllergyIntolerance.

Trabajo Futuro

- Migrar la orquestación a Kubernetes para pruebas con nodos en diferentes regiones de la nube.
- Implementar replicación nativa de PostgreSQL mediante Patroni/etcd para garantizar consistencia ACID a nivel de motor.
- Extender la conformidad FHIR hacia una certificación completa con el servidor HAPI FHIR oficial.

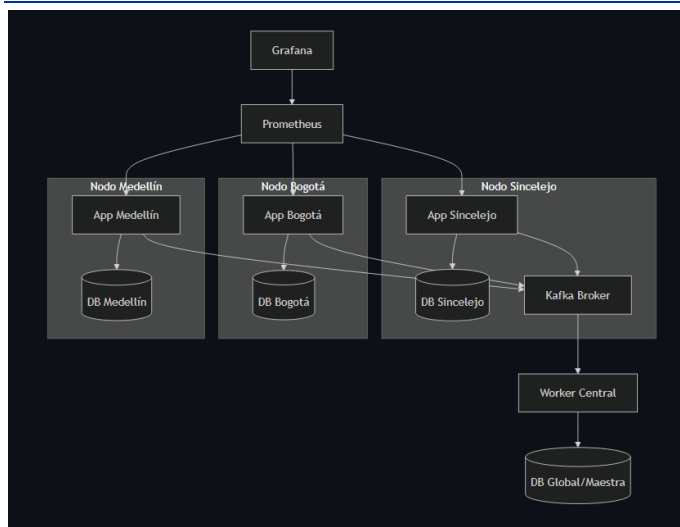


Ilustración 1: [Diagrama de Arquitectura]

Cada nodo expone endpoints compatibles con FHIR R4, incluyendo un CapabilityStatement en /api/v1/fhir/metadata, y recursos clínicos como Patient, Observation y Condition. La orquestación completa del stack se gestiona mediante Docker Compose, con health-checks y restart policies definidos para cada servicio.

B. Flujo de Datos y Sincronización

Cuando un evento clínico (registro de paciente, triage, diagnóstico) se genera en una sede, el servidor publica un mensaje en el topic eventos-clínicos de Kafka. Un Worker Central suscrito a dicho topic consume los eventos y los replica en todos los nodos restantes. Este mecanismo garantiza consistencia eventual en menos de 2 segundos bajo condiciones normales de red.

C. Tolerancia a Fallos (Offline-First)

Ante la pérdida de conectividad con Kafka, el nodo afectado persiste los eventos localmente en un archivo pendientes [sede].json. Un proceso de sincronización automática ejecutado cada 15 segundos intenta republicar los eventos acumulados al recuperar la conexión. Adicionalmente, si la base de datos local falla, el servidor conmuta automáticamente a las bases de datos de los nodos alternativos, garantizando operación continua.

D. Seguridad

La autenticación y autorización se implementan bajo el protocolo SMART on FHIR. Cada petición requiere un token JWT firmado con HMAC-SHA256, validado por el middleware verificarTokenFHIR. Los endpoints clínicos son inaccesibles sin un token válido con el scope correspondiente.

E. Observabilidad

Cada nodo expone un endpoint /metrics con métricas en formato Prometheus: uptime, total de peticiones, errores y activaciones de failover. Un dashboard en Grafana (puerto 3005) consolida estas métricas en tiempo real para los tres nodos.

- Incorporar un modelo de Zero Trust completo siguiendo las recomendaciones de Alsubaei et al. [3], con rotación automática de tokens y auditoría de acceso por recurso.

VII. CONCLUSIONES

1. La arquitectura orientada a eventos es viable para redes de salud distribuidas. El uso de Apache Kafka como bus de mensajería demostró que es posible mantener la consistencia clínica entre nodos geográficos sin depender de una conexión permanente, lo que representa una solución realista para el contexto latinoamericano donde la conectividad es intermitente.
2. El estándar HL7 FHIR R4 es implementable sin infraestructura especializada. La emulación de endpoints FHIR sobre Node.js demostró que instituciones con recursos tecnológicos limitados pueden adoptar este estándar de interoperabilidad sin necesidad de desplegar servidores especializados como HAPI FHIR, reduciendo significativamente la barrera de entrada.
3. La contenerización es un habilitador crítico para la reproducibilidad. Docker Compose permitió que el stack completo de tres nodos, con todas sus dependencias, fuera desplegable en cualquier máquina con un solo comando, eliminando los errores de configuración típicos de instalaciones manuales y facilitando la adopción por parte de equipos técnicos con distintos niveles de experiencia.
4. La seguridad SMART on FHIR es esencial, no opcional. La implementación de OAuth2 con JWT como capa transversal demostró que proteger los datos clínicos no requiere soluciones complejas; un middleware bien diseñado garantiza que ningún endpoint sea accesible sin autorización explícita, cumpliendo los principios de privacidad del paciente.
5. Este prototipo sienta las bases para una red nacional de salud interoperable. Escalar este modelo a infraestructura real en la nube, incorporando replicación nativa de bases de datos y orquestación con Kubernetes, representa un camino técnico claro hacia un sistema de historia clínica electrónica unificado que podría impactar positivamente la calidad y continuidad de la atención médica en Colombia.

REFERENCIAS

- [1] A. Kiourtis et al., *J. Biomed. Inform.*, vol. 119, 2021.
- [2] T. Müller et al., *IEEE Access*, vol. 10, 2022.
- [3] F. Alsubaei et al., *Internet of Things*, vol. 8, 2019.
- [4] G. Aceto et al., *J. Ind. Inf. Integr.*, vol. 18, 2020.
- [5] J. Torres-Sánchez y C. Ramírez, *Rev. Panam. Salud Pública*, vol. 46, 2022.